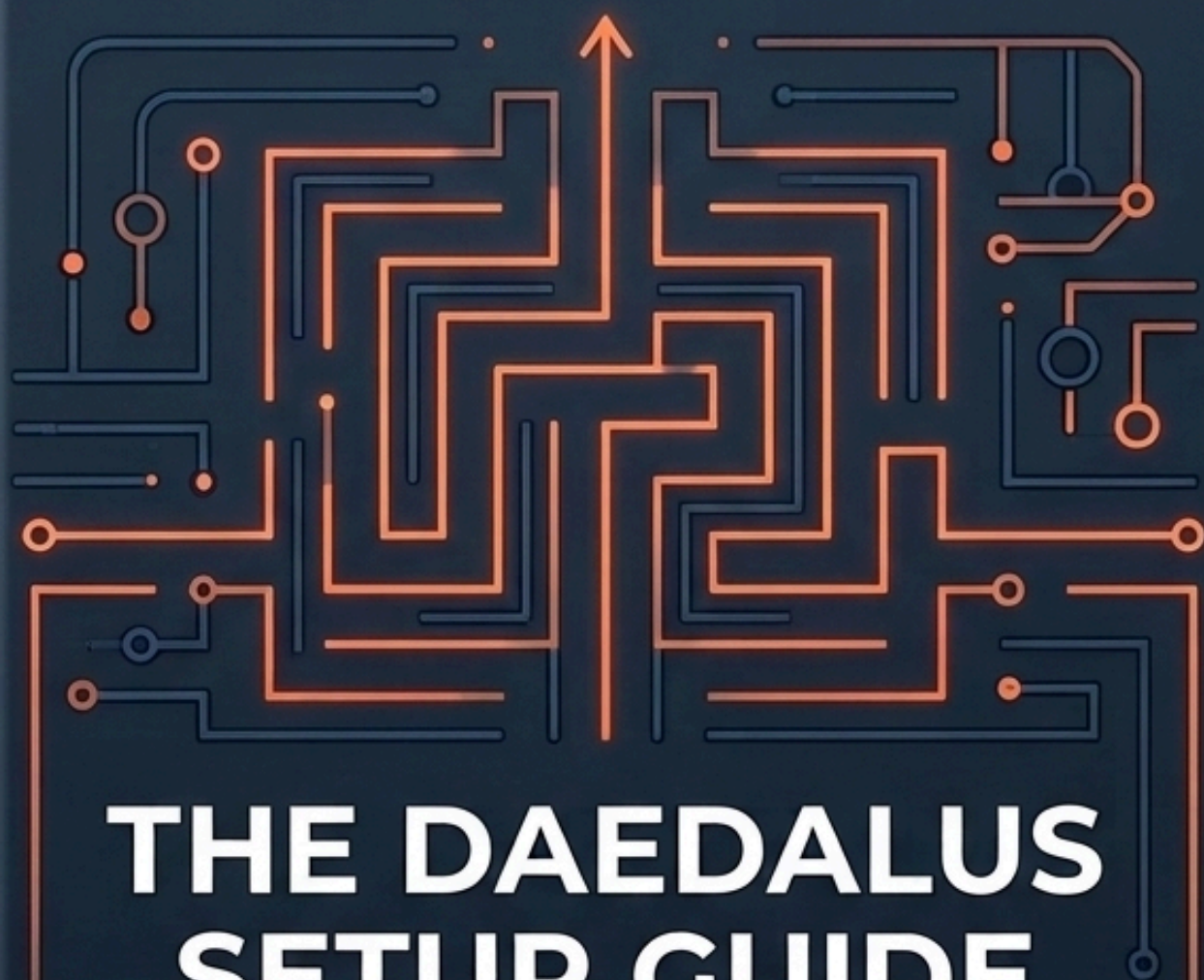


NAVIGATE THE AI LABYRINTH



THE DAEDALUS SETUP GUIDE

A Comprehensive Developer Manual
for Building & Deploying AI Systems

BY BRIAN GILL
AUTHOR OF "CODE & THE MACHINE"

Navigate the AI Labyrinth: The Daedalus Setup Guide

A Premium Developer Kit for Custom AI Coding Assistants

Version: 1.0.0

Published: July 2026

Author: Brian Gill

License: MIT / Branded Commercial

Table of Contents

1. Overview & Architecture
 2. Prerequisites & Setup
 3. The InMemory Router
 4. The Interactive REPL Shell
 5. Branding & Customization Guidelines
 6. Extending the CLI
 7. Monetization & Pricing Models
 8. NPM Publishing & Releases
-

1. Overview & Architecture

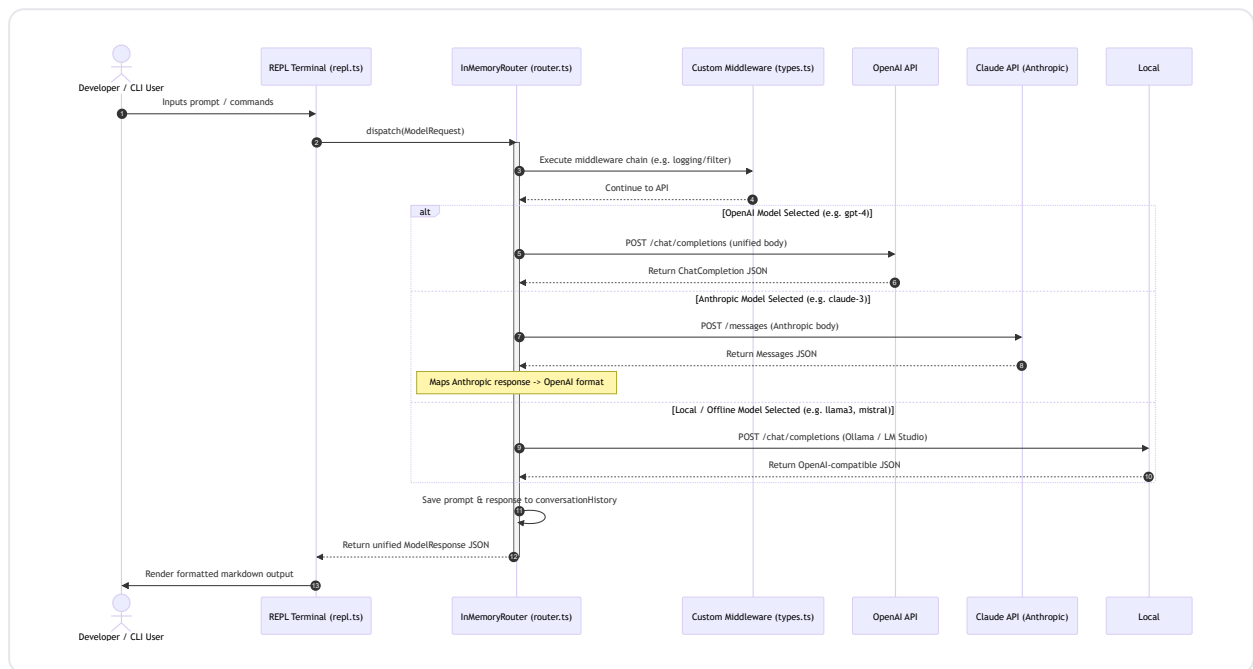
1.1 What is Daedalus-Lite?

Daedalus-Lite is a lightweight, opinionated, zero-dependency starter template for building and packaging your own branded AI-powered command-line interfaces. It provides an in-memory API router, a command line interface (REPL) using Node's native readline module, and a strict TypeScript layout. This template allows developers to create

a customized pair-programming terminal assistant, package it under their own brand, and sell it directly to users or clients.

1.2 System Architecture

The application runs entirely locally on the user's terminal, communicating directly with API endpoints (OpenAI or Anthropic) without any intermediary tracking servers.



2. Prerequisites & Setup

2.1 System Requirements

To build and run the template, ensure your system meets the following specifications:

- **Node.js:** `v20.x` or higher (necessary for native `fetch` and ES Module loading).
- **npm:** `v10.x` or higher.
- **TypeScript:** `v5.x` or higher (installed as a development dependency).

2.2 Quick Start Installation

1. Initialize the project directory and install dependencies:

```
npm install
```

2. Build the TypeScript source code:

```
npm run build
```

3. Link the package globally to your system terminal:

```
npm link
```

4. Run the interactive REPL:

```
npm run repl
```

Or start the CLI directly:

```
npm start
```

3. The InMemory Router

The core router (`src/router.ts`) is designed to handle multiple AI endpoints dynamically, mapping responses into a unified structure.

3.1 Mapping Schema

The router maps the different response bodies of OpenAI and Anthropic into a single unified `ModelResponse` interface:

```
export interface ModelResponse {  
  id: string;  
  object: string;  
  created: number;
```

```

choices: Array<{
  index: number;
  message: { role: string; content: string; };
  logprobs: null | any;
  finish_reason: string;
}>;
usage: {
  prompt_tokens: number;
  completion_tokens: number;
  total_tokens: number;
};
}

```

This guarantees that your front-end or REPL loops do not need to care which model is running; the response format remains identical.

3.2 Local & Offline AI Integration (Ollama & LM Studio)

By default, any model specified in the REPL (e.g. `/model llama3` or `/model deepseek-coder`) that does not start with `gpt-` or `claude-` is automatically routed as a local model. Local engines host OpenAI-compatible `/v1/chat/completions` API servers on your machine. The router queries these endpoints directly without requiring external keys:

- **Ollama (Default):** Exposes endpoints on `http://localhost:11434/v1`.
- **LM Studio:** Exposes endpoints on `http://localhost:1234/v1`.

You can configure your local engine endpoint URL in your environment:

```
LOCAL_ENDPOINT_URL=http://localhost:11434/v1
```

Or pass it directly in the router config:

```

const router = new InMemoryRouter({
  localEndpointUrl: 'http://localhost:11434/v1',
  defaultModel: 'gpt-4',
  temperature: 0.7,
});

```

4. The Interactive REPL Shell

The REPL (`src/repl.ts`) creates a readline session that reads inputs line-by-line.

4.1 Built-in Commands

The REPL recognizes the following command inputs prefixed with a `/` :

- `/help` - Displays the command manual.
- `/model <modelName>` - Swaps the active routing model (e.g., `/model gpt-4`).
- `/clear` - Empties the current session's conversation history.
- `/exit` - Terminates the process.

5. Branding & Customization Guidelines

The power of `daedalus-lite` is that you can completely rebrand the tool for your target audience.

5.1 Color Tokens (CSS & TUI)

For web-based visual consoles, CLI text decorators, or marketing PDF documents, use the following brand tokens:

Token	Hex Value	Tailwind Class	Usage
Primary Blue	#1E3A8A	<code>bg-sky-600</code> , <code>text-sky-600</code>	Headers, links, borders, branding buttons
Accent Orange	#F59E0B	<code>bg-amber-500</code> , <code>text-amber-500</code>	Critical highlights, warnings, call-to-actions
Background Gray	#F3F4F6	<code>bg-slate-100</code>	Console panels, backdrop frames
Primary Text	#111827	<code>text-slate-900</code>	Code output, main content

5.2 Voice & Tone

When customizing prompt responses or error warnings, keep the voice **approachable, technical, and direct**. Avoid using emoji in standard system prints, and ensure errors provide actionable recovery tips.

5.3 Branded Assets — Swap, Don't Hunt

The template ships with placeholder images in `docs/manual/` . Replace them with your own files — same filenames — and everything still works out of the box:

File	Used By
<code>daedalus_guide_cover.png</code>	PDF cover page (<code>compile_pdf.js</code>)
<code>daedalus_lite_logo.png</code>	Landing page & hero section
<code>daedalus_evolution_infographic.jpg</code>	Setup manual body

No config files to edit. No paths to update. Just drop in your PNG/JPG with the same filename and re-run the PDF compiler.

6. Extending the CLI

6.1 Creating Custom Middleware

You can easily inject custom middleware into the request routing pipeline. This is perfect for adding logs, usage rate limits, or custom system prompts.

```
// Example: System Prompt Injector Middleware
import type { Middleware } from './types.js';

export const systemPromptMiddleware: Middleware = async (request, next) => {
  request.prompt = `Act as an expert software architect. ${request.prompt}`;
  return next();
};
```

Register your custom middleware by calling the router's `.use(middleware)` method:

```
router.use(systemPromptMiddleware);
```

7. Monetization & Pricing Models

Selling your customized CLI can follow three distinct monetization tracks:

Tier	Price Point	Deliverable	Target Audience
The Blueprint PDF	\$19 - \$29	High-quality setup manual PDF + GitHub repository template access.	Indie hackers, junior developers
Branded CLI Package	\$49 - \$99	Fully pre-configured CLI compiled binaries, custom banners, custom theme.	Small developer teams
CTO Integration Call	\$249 - \$499	The manual + the branded code + a 1-hour live setup and onboarding call.	Corporate teams, agencies

8. NPM Publishing & Releases

Once you are ready to publish your template:

1. Update `package.json` with your custom name:

```
{
  "name": "your-branded-cli",
  "version": "1.0.0",
  "type": "module",
  "main": "dist/index.js"
}
```


2. Build the TypeScript source:

```
npm run build
```

3. Publish to the public NPM registry:

```
npm publish --access public
```

4. To package the CLI into standalone executable files (no Node.js installation required for the end-user), use package compilers like `pkg` :

```
npx pkg . --targets node20-win-x64,node20-macos-x64,node20-linux-x64
```

Navigate the AI Labyrinth with confidence.

© 2026 Brian Gill. All rights reserved.